Microservices Inventory Management System

Event-Driven, Fault-Tolerant, Scalable Retail Backend

LogicVeda Web Development Domain Capstone Project - March 2026

Prepared for final project submission.

1. Project Overview

Microservices Inventory Management System is a production-style inventory and order management platform for e-commerce and retail operations. It decomposes authentication, inventory, orders, reporting, event delivery, and gateway responsibilities into independently deployable services.

The platform demonstrates service decomposition, eventual consistency through sagas, API gateway routing, JWT authorization, durable event delivery, observability, resilience patterns, Docker local deployment, and Kubernetes-ready production deployment.

Target Users

- E-commerce startups that need reliable catalog, stock, and order APIs.
- Retail teams that need admin inventory visibility and low-stock alerts.
- Supply-chain SaaS platforms that need an integration-ready backend.

Non-Functional Goals

- Throughput target: at least 1000 requests per minute.
- Resilience: circuit breakers, retries, timeouts, rate limiting, and bulkheads.
- Observability: structured logs, metrics, Prometheus output, Grafana dashboards, and correlation IDs.
- Data consistency: saga-based eventual consistency instead of distributed ACID transactions.

2. Key Features

ID	Feature	Description	Acceptance Criteria
F01	API Gateway	Custom NestJS gateway for service routing, auth proxying, rate limiting, health, and metrics.	All API requests route correctly and JWT validation is enforced for protected paths.
F02	Authentication Service	Register, login, token verification, and role management.	JWT issuance and validation work across services with Admin and Staff roles.
F03	Inventory Service	Product CRUD, stock levels, low-stock alerts, atomic stock adjustments, and optimistic locking.	Stock updates are atomic and concurrent writes are guarded by version checks.
F04	Orders Service	Order creation, status tracking, payment simulation, and saga orchestration.	Order creation triggers stock reservation and reaches confirmed or failed status.
F05	Event Bus	Durable event log, retry delivery, audit view, idempotent consumers, and optional Kafka mirroring.	Events are delivered at least once and duplicate side effects are prevented.
F06	Reporting and Search	Sales, inventory, stock-alert, and event projections with PostgreSQL facets and optional OpenSearch.	Aggregations, faceted search, and realtime stock metrics are available.
F07	Resilience Patterns	Gateway rate limits, upstream timeouts, safe-method retries, circuit breakers, and bulkheads.	Partial failures degrade gracefully rather than cascading across the system.
F08	Observability	Structured JSON logs, health checks, metrics, Prometheus endpoint, Grafana provisioning, and Jaeger profile.	Requests carry correlation IDs and operational state is observable.

3. Technology Stack

Category	Technology	Rationale
Frontend	React, TypeScript, TailwindCSS, Redux Toolkit	Typed, responsive admin UI and public catalog.
Backend	Node.js 22, NestJS	Structured services, dependency injection, validation, and Swagger support.
Gateway	Custom NestJS API Gateway	Direct control over auth proxying, routing, metrics, and resilience behavior.
Data	PostgreSQL, Prisma	ACID service databases with versioned migrations and typed access.
Search	OpenSearch optional, PostgreSQL fallback facets	Production search can use OpenSearch while local deployments remain self-contained.
Messaging	Durable HTTP event bus, optional Kafka mirror	Runnable by default, with Kafka available for production event streaming.
Cache	Redis	Distributed gateway rate limiting.
Observability	Prometheus, Grafana, Jaeger profile, JSON logs	Operational visibility and request correlation.
Deployment	Docker Compose, Kubernetes manifests, GitHub Actions	Local repeatability and production-ready orchestration.

4. Architecture

```
React Admin UI / Public Catalog
          |
          v
API Gateway - JWT validation, routing, rate limits, resilience
   |        |          |          |            |
   v        v          v          v            v
```

```
Auth     Inventory   Orders    Reporting    Event Bus
 DB         DB        DB          DB       Durable Event Log
                         |          ^
                         v          |
                  OrderCreated -> StockSucceeded / StockFailed
                         |
                         v
                Saga confirmation or compensation
```

Each service owns its database. The order workflow uses events and idempotent consumers to maintain eventual consistency between orders, stock reservations, and reporting projections.

5. Execution Timeline

Phase	Deliverables
Week 1	Service map, monorepo, authentication, inventory CRUD, gateway routing, Docker Compose, smoke tests.
Week 2	Event bus, order service, saga workflow, event consumers, admin frontend, end-to-end order placement.
Week 3	Resilience patterns, product search, faceted filters, reporting dashboards, structured logs, correlation IDs, load testing.
Week 4	Kubernetes manifests, ingress/HPA, GitHub Actions, observability dashboards, final QA, README, report export.

6. Technical Highlights

Security

- JWT bearer authentication at the gateway and service level.
- Role-based authorization for Admin and Staff workflows.
- DTO validation, whitelisting, and non-whitelisted field rejection.

- Helmet, CORS configuration, and no committed `.env` requirement.

Performance and Reliability

- Gateway rate limiting backed by Redis.
- Safe-method retries and upstream timeouts.
- Circuit breaker and per-upstream bulkhead protections.
- Load test result: 1000 requests, 0 failures, more than 1000 requests per minute.

Data Consistency

- Orders publish `OrderCreated` events after simulated payment authorization.
- Inventory reserves stock atomically and publishes `StockSucceeded` or `StockFailed`.
- Order service confirms or fails orders based on stock outcomes.
- Processed event tables prevent duplicate side effects.

7. Deployment and Operations

The project can run locally with Docker Compose and is prepared for Kubernetes deployment. The Kubernetes manifest defines services, deployments, probes, resources, ingress, secrets, configuration, migrations, Redis, and HPAs.

For public deployment, provide a container registry, production database URLs, Redis URL, production secrets, DNS access, and a TLS strategy. The detailed checklist is maintained in `docs/DEPLOYMENT_RUNBOOK.md`.

Operational Endpoints

- Gateway health: `/health`
- Gateway metrics: `/metrics`
- Prometheus metrics: `/metrics/prometheus`
- Service Swagger docs: `/docs` on each backend service

8. Visuals and Demo Plan

Recommended screenshots or video scenes for final submission:

1. Public catalog without login.
2. Login page with demo credentials.
3. Products table and product creation form.
4. Atomic stock adjustment and low-stock state.
5. Order creation and order status history.
6. Reports dashboard with sales and inventory projections.
7. Gateway health and metrics endpoints.
8. Docker/Kubernetes deployment status.

9. API and Workflow Summary

The system exposes a REST API through the gateway. The gateway validates bearer tokens for protected routes and forwards requests to the correct upstream service. Public routes include login, registration, token verification, health, and the public product catalog.

Workflow	Primary Endpoints	Expected Result
Authentication	`POST /auth/login`, `POST /auth/register`, `POST /auth/verify`	JWT token issuance, validation, and role-aware user context.
Product Catalog	`GET /products`, `POST /products`, `PUT /products/:id`, `DELETE /products/:id`	Admin-managed catalog with pagination, search, category filters, and facets.
Stock Control	`PUT /inventory/:id/stock`, `POST /inventory/bulk-update`	Atomic stock adjustments with optimistic locking and low-stock events.
Order Processing	`POST /orders`, `GET /orders`, `GET /orders/:id`, `PUT /orders/:id/status`	Order creation, simulated payment authorization, status history, and saga confirmation.
Reporting	`GET /reports/sales`, `GET /reports/inventory`, `GET`	Sales totals, inventory snapshots, stock alerts, and event audit data.

Workflow	Primary Endpoints	Expected Result
	`/reports/stock-alerts`, `GET /reports/events`	

The primary end-to-end business path is: an Admin creates a product, Staff or Admin creates an order, the order service authorizes simulated payment, the event bus publishes `OrderCreated`, inventory reserves stock, order status becomes `CONFIRMED`, and reporting receives updated sales and inventory projections.

10. Testing and Quality Evidence

The project includes unit tests around gateway routing, authentication response safety, event bus retry behavior, inventory stock events, order utility calculations, order event handling, reporting projections, and reporting utility functions. The frontend currently relies on TypeScript build validation and production bundle checks.

Command	Purpose	Latest Result
`npm run verify`	Prisma generation, full build, and all workspace tests.	Passed.
`npm run lint`	ESLint checks across services, frontend, and packages.	Passed.
`docker compose config --quiet`	Validates Docker Compose syntax and interpolation.	Passed.
`./scripts/smoke-test.sh`	Creates a temporary product, places an order, waits for saga confirmation, and cleans up.	Passed.
`npm run load:test`	Checks gateway throughput against the PDF target.	1000 requests, 0 failures, more than 1000 requests per minute.

A first load-test attempt immediately after smoke testing hit the configured gateway rate limit.

After the rate-limit window reset, the same 1000-request test passed cleanly. This confirms the failures were controlled rate limiting rather than service instability.

11. Production Readiness and Deployment Inputs

The repository is ready for a public deployment once external account information is available. The codebase cannot complete these items by itself because they depend on the owner's cloud, DNS, registry, and secret-management choices.

Required Input	Why It Is Needed
Container registry namespace	All service images must be pushed before Kubernetes can pull them.
Public deployment target	The live demo must run on a public platform or Kubernetes cluster.
Domain and DNS access	The PDF requires a public HTTPS demo URL.
Production PostgreSQL URLs	Each database-backed service needs runtime and migration connection strings.
Redis URL	Required for distributed gateway rate limiting in production.
JWT and internal event secrets	Required for secure user tokens and internal event delivery.
Kafka and OpenSearch URLs	Optional production-grade event streaming and search backends.

The detailed deployment instructions and exact environment variable names are maintained in `docs/DEPLOYMENT_RUNBOOK.md`.

12. Reflection and Roadmap

The project applies production web engineering patterns including microservice boundaries,

asynchronous event delivery, idempotent consumers, gateway-level resilience, and observability. Future improvements include managed Kafka as the primary event transport, managed OpenSearch in production, richer warehouse/location modeling, and automated screenshot generation for release evidence.